Paul Belleville - INALCO (ERTIM)

# Programmation Python --- Séance 5

## Les boucles : répéter des instructions

**Durée indicative :** 3 heures
**Thème :** boucles `while` et `for`, répétition, compteurs et conditions
**Support d'exercices et d'apprentissage :** Python Programming MOOC --- Université d'Helsinki

---

## 1. Objectifs de la séance

À la fin de cette séance, vous devez être capables de :

- expliquer à quoi sert une boucle dans un programme ;
- utiliser une boucle `while` pour répéter des instructions tant qu'une condition est vraie ;
- utiliser une boucle `for` avec `range()` pour répéter des instructions un nombre déterminé de fois ;
- combiner une boucle et une condition (`if`) ;
- utiliser une variable compteur pour suivre le nombre de répétitions;
- repérer les causes courantes d'une boucle infinie.

### Prérequis

Cette séance s'appuie sur les notions suivantes :

- variables et types de données ;
- saisie et affichage avec `input()` et `print()` ;
- conversions de types (`int()`, `float()`) ;
- conditions (`if`, `elif`, `else`) ;
- opérateurs de comparaison et opérateurs logiques ;
- fonctions et chaînes de caractères.

---

## 2. Pourquoi utiliser des boucles ?

Une boucle permet de **répéter un bloc d'instructions** sans avoir à écrire plusieurs fois le même code.

Imaginons que nous souhaitions afficher les nombres de 1 à 5. Sans boucle, nous pourrions écrire :

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

Cela fonctionne, mais le programme devient vite répétitif. Que faudrait-il faire pour afficher les nombres de 1 à 100, ou de 1 à 1 000 ?

Avec une boucle :

```
for i in range(1, 6):
    print(i)
```

Résultat :

```
1
2
3
4
5
```

Le bloc `print(i)` est exécuté plusieurs fois. À chaque répétition, la valeur de `i` change.

**À retenir :** une boucle est utile lorsqu'une opération doit être répétée, que ce soit un nombre de fois connu à l'avance ou jusqu'à ce qu'une condition soit remplie.

---

# 3. La boucle `while`

## 3.1. Principe

La boucle `while` répète un bloc d'instructions **tant que sa condition est vraie**.

Syntaxe générale :

```
while condition:
    instruction_1
    instruction_2
```

Exemple :

```
number = 1

while number < 5:
    print(number)
    number = number + 1

print("Fin du programme")
```

Résultat :

```
1
2
3
4
Fin du programme
```

Voici ce qui se passe :

1. La variable `number` est initialisée à `1`.
2. Python vérifie si `number < 5`.
3. Si la condition est vraie, le contenu de la boucle est exécuté.
4. La variable `number` augmente de `1`.
5. Python revient vérifier la condition.
6. Lorsque `number` vaut `5`, la condition devient fausse : la boucle s'arrête.

### 3.2. Les trois éléments importants d'une boucle `while`

Dans beaucoup de boucles `while`, on retrouve trois éléments :

- **Initialisation :** donner une première valeur à la variable utilisée dans la condition.
- **Condition :** déterminer si la boucle doit continuer.
- **Mise à jour :** modifier une variable pour que la condition puisse éventuellement devenir fausse.

Exemple :

```
number = 1                  # Initialisation

while number <= 10:         # Condition
    print(number)
    number = number + 1     # Mise à jour
```

Si la variable n'est jamais modifiée, la condition peut rester vraie indéfiniment.

### 3.3. Attention aux boucles infinies

Une boucle infinie est une boucle qui ne s'arrête jamais d'elle-même.

Exemple :

```
number = 1

while number < 5:
    print(number)
```

La valeur de `number` ne change jamais. Elle reste égale à `1`, donc la condition `number < 5` reste toujours vraie.

Pour interrompre un programme bloqué dans le terminal, on peut généralement utiliser **Ctrl + C**.

**Conseil :** lorsque vous écrivez une boucle `while`, demandez-vous toujours : « Qu'est-ce qui va faire que la condition deviendra fausse ? »

### 3.4. Répéter une saisie jusqu'à obtenir la bonne réponse

Une boucle `while` est particulièrement utile lorsque l'on ne connaît pas à l'avance le nombre de répétitions nécessaires.

```
password = ""

while password != "python123":
    password = input("Mot de passe : ")

print("Accès autorisé")
```

Le programme demande un mot de passe tant que celui saisi est différent de `"python123"`.

Si l'utilisateur saisit immédiatement le bon mot de passe, le bloc de la boucle n'est pas exécuté : la condition est vérifiée **avant** chaque répétition.

---

## 4. La boucle `for` et la fonction `range()`

### 4.1. Principe

La boucle `for` permet notamment de répéter un bloc d'instructions pour chaque valeur d'un **itérable**.

Pour commencer, nous allons l'utiliser avec `range()`, qui génère une suite de nombres.

```
for i in range(5):
    print(i)
```

Résultat :

```
0
1
2
3
4
```

Attention : `range(5)` commence à `0` et s'arrête **avant** `5`. La valeur finale n'est donc pas incluse.

### 4.2. Définir un début et une fin

On peut préciser le début et la fin de la séquence :

```
for i in range(1, 6):
    print(i)
```

Résultat :

```
1
2
3
4
5
```

La syntaxe `range(debut, fin)` inclut le début, mais exclut la fin.

### 4.3. Définir un pas

On peut également préciser l'écart entre deux valeurs successives :

```
for i in range(0, 10, 2):
    print(i)
```

Résultat :

```
0
2
4
6
8
```

Ici, `range(0, 10, 2)` signifie : commencer à `0`, s'arrêter avant `10` et avancer de `2` à chaque étape.

Autres exemples :

```
range(5)         # 0, 1, 2, 3, 4
range(1, 6)      # 1, 2, 3, 4, 5
range(0, 10, 2) # 0, 2, 4, 6, 8
```

### 4.4. `for` ou `while` : lequel choisir ?

Les deux structures permettent de répéter des instructions, mais elles ne répondent pas exactement aux mêmes besoins.

- On utilise souvent `for` lorsque l'on sait combien de répétitions effectuer, ou lorsque l'on parcourt une séquence.
- On utilise souvent `while` lorsque la répétition dépend d'une condition et que l'on ne connaît pas à l'avance le nombre de répétitions.

Exemple avec `for` :

```
for i in range(5):
    print("Bonjour")
```

Exemple avec `while` :

```
answer = ""

while answer != "oui":
    answer = input("Voulez-vous continuer ? ")

print("Très bien !")
```

Il s'agit d'une règle pratique, pas d'une interdiction : certains problèmes peuvent être résolus avec les deux types de boucles.

---

## 5. Combiner les boucles et les conditions

Une boucle peut contenir une condition `if`. La boucle contrôle **combien de fois** les instructions sont répétées ; la condition permet de décider **quoi faire** à chaque répétition.

Exemple : afficher uniquement les nombres pairs entre 1 et 10.

```
for i in range(1, 11):
    if i % 2 == 0:
        print(i)
```

Résultat :

```
2
4
6
8
10
```

L'opérateur `%` *("modulo")* donne le reste d'une division. Un nombre est pair lorsque son reste dans une division par `2` est égal à `0`.

On peut également utiliser une condition dans une boucle `while` :

```
number = 1

while number <= 10:
    if number % 2 == 0:
        print(number)

    number = number + 1
```

Le résultat est le même.

## 5.1. Les compteurs

Une variable compteur permet de compter le nombre de fois qu'un événement se produit ou qu'une boucle est exécutée.

```
attempts = 0

for i in range(3):
    attempts = attempts + 1
    print("Tentative numéro", attempts)

print("Nombre total de tentatives :", attempts)
```

On peut aussi écrire `attempts += 1`, ce qui signifie la même chose que `attempts = attempts + 1`.

## 5.2. Les accumulateurs

Une variable peut également servir à accumuler un résultat, par exemple une somme.

```
total = 0

for number in range(1, 4):
    total = total + number

print(total)
```

Résultat :

```
6
```

À chaque répétition, on ajoute la valeur courante de `number` à `total`.

---

# 6. Exercices --- MOOC de l'Université d'Helsinki

Les exercices sont réalisés directement sur la plateforme du MOOC. Celle-ci exécute des tests automatiques et vous indique si votre solution correspond au résultat attendu.

**Consignes :**

1. Lisez attentivement l'énoncé et les exemples d'entrée/sortie.
2. Essayez de comprendre le résultat attendu avant de commencer à coder.
3. Écrivez votre solution dans l'éditeur de la plateforme.
4. Exécutez les tests et lisez les éventuels messages d'erreur.
5. Si un exercice vous bloque, passez au suivant et revenez-y plus tard.

Il n'est pas nécessaire de terminer tous les exercices pendant la séance. Privilégiez la compréhension et les exercices réalisés correctement.

## Série A --- Découvrir les boucles

Lien : Part 2 --- Simple loops

Exercices à réaliser dans l'ordre :

1. **Shall we continue?** --- répéter une action jusqu'à ce que l'utilisateur saisisse `no`.
2. **Input validation** --- demander des nombres et réagir différemment selon leur valeur.
3. **Fix the code: Countdown** --- corriger une boucle qui ne fonctionne pas comme prévu.
4. **Repeat password** --- demander à l'utilisateur de confirmer un mot de passe.

## Série B --- Boucles et conditions

Lien : Part 3 --- Loops with conditions

Exercices à réaliser si vous avez terminé la série A :

1. **Print numbers** --- afficher une suite de nombres.
2. **Fix the code: Countdown** --- corriger un programme de décompte.
3. **Numbers** --- travailler avec des valeurs numériques et des conditions.
4. **Powers of two** --- calculer des puissances de deux à l'aide d'une boucle.
5. **The sum of consecutive numbers** --- calculer une somme à partir d'une suite de nombres.

## Pour aller plus loin

Vous pouvez poursuivre avec la section More loops, qui propose notamment des exercices sur la multiplication et les factorielles.

---

# 7. Mini-projet : Jeu de devinette

Nous allons construire un petit jeu dans lequel l'utilisateur doit deviner un nombre choisi à l'avance.

## Objectif

Le programme possède un nombre secret. Il demande à l'utilisateur de proposer un nombre et lui indique si sa proposition est trop petite ou trop grande. Il continue jusqu'à ce que l'utilisateur trouve le nombre.

## Étape 1 : Faire une seule tentative

Pour commencer, le nombre secret est fixé à `42`.

```
secret = 42
guess = int(input("Devinez le nombre : "))

if guess == secret:
    print("Bravo !")
elif guess < secret:
    print("Trop petit !")
else:
    print("Trop grand !")
```

Testez le programme avec plusieurs valeurs.

## Étape 2 : Répéter les tentatives

Nous voulons maintenant que l'utilisateur puisse recommencer tant qu'il n'a pas trouvé le nombre.

```
secret = 42
guess = -1

while guess != secret:
    guess = int(input("Devinez le nombre : "))

    if guess < secret:
        print("Trop petit !")
    elif guess > secret:
        print("Trop grand !")

print("Bravo, vous avez trouvé !")
```

Pourquoi la variable `guess` est-elle initialisée à `-1` avant la boucle ?

## Étape 3 : Compter les tentatives

Ajoutons une variable qui compte le nombre de propositions faites par l'utilisateur.

```
secret = 42
guess = -1
attempts = 0

while guess != secret:
    guess = int(input("Devinez le nombre : "))
```

```
    attempts = attempts + 1

    if guess < secret:
        print("Trop petit !")
    elif guess > secret:
        print("Trop grand !")

print("Bravo, vous avez trouvé !")
print("Nombre de tentatives :", attempts)
```

### Défis supplémentaires

Si vous avez terminé les trois étapes précédentes, essayez d'améliorer le jeu :

- limiter le nombre de tentatives à 10
- afficher un message différent selon le nombre de tentatives
- permettre à l'utilisateur de recommencer une nouvelle partie
- remplacer le nombre fixe `42` par un nombre choisi aléatoirement.

Pour le dernier défi, vous pouvez utiliser le module `random` :

```
import random

secret = random.randint(1, 100)
```

La fonction `randint(1, 100)` choisit un entier aléatoire entre 1 et 100, bornes incluses.

---

## 8. Bilan --- Ce qu'il faut retenir

- Une boucle permet de répéter un bloc d'instructions.
- `while` répète les instructions tant que sa condition est vraie.
- Une boucle `while` doit pouvoir atteindre une condition d'arrêt ; sinon, elle risque de devenir infinie.
- `for` permet notamment de parcourir les valeurs produites par `range()`.
- `range(fin)` commence à `0` et s'arrête avant `fin`.
- `range(debut, fin, pas)` permet de préciser le début, la fin et le pas.
- Une condition `if` peut être placée à l'intérieur d'une boucle.
- Un compteur sert à compter ; un accumulateur sert à construire un résultat au fil des répétitions.

### Questions de vérification

1. Quelle est la différence entre `for` et `while` ?

   ▶ Réponse

   ```
   La boucle for s'utilise lorsque l'on connaît le nombre de répétitions
   à l'avance (que ce soit un nombre donné ou un objet itérable), tandis
   ```

```
que la boucle while se répète indéfiniment tant qu'une condition reste
vraie.
```

2. Que va afficher le programme suivant ?

```
for i in range(1, 5):
    print(i)
```

▶ Réponse

3. Pourquoi le programme suivant ne s'arrête-t-il jamais ?

```
number = 1

while number < 5:
    print(number)
```

▶ Réponse

```
 Car rien dans le code éxecuté par la boucle ne permet de rendre la
condition fausse. La condition reste `true`, donc la boucle continue
de s'éxecuter.
```

4. Que fait l'opérateur `%` dans l'exemple qui affiche les nombres pairs ?

▶ Réponse

```
 L'opérateur % (modulo) calcule le reste de la division entière, ce
qui permet de vérifier si un nombre est pair en testant si nombre % 2
== 0 (le reste est égal à zéro).
 Si le reste est égal à 0, alors c'est un nombre pair.
```

5. À quoi sert la variable `attempts` dans le mini-projet ?

▶ Réponse

---

## Ressources

- Python Programming MOOC 2026 --- Université d'Helsinki
- Part 2 --- Simple loops
- Part 3 --- Loops with conditions
- Part 3 --- More loops